

Process & Decision Documentation

Side Quests and A4 (Individual Work)

Focus on:

One significant decision or change you made? Why you made it? What effect it had on the work?

I started off with a very complicated idea, and had to scope my vision down into something that I could code. I made the different options more limited, keeping it to two questions instead, while keeping a story based line. This made the code visually appealing but also fun without becoming too plain and boring. Using text, buttons and colors made my work more appealing for the code.

GenAI Documentation

Date Used: Jan.30.2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Debugging and code structure

Summary of Interaction: Guided in hover effects, generated the structure of code

Human Decision Point(s): Overruled number key pressed; GenAI added without my consent. I changed the order of flies in my code; the structure in the index was not matched with GenAI's since it didn't know the order they were ever in. I added a box behind the buttons so that it wasn't just boring text like it was initially given to me.

Integrity & Verification Note: gained knowledge of basic code through first year I was able to understand and read over the code to make sure I knew what certain functions and lines were capable of.

Scope of GenAI Use:

The structure of how the code was going to flow including the overarching lunch time idea including questions, responses and choices were all made by

me. Design choices of the hover, button and color scheme were also all designed by me. I edited the Index on my own, adding to the work that was already in that file since GenAI did not know the order and was not making the code work in a flow state with flies.

Limitations or Misfires: the flies not being on the same page confused the flow of the code and structure. I had to ask for the code in one long line so that I could see if it worked in the first place and then edited

Summary of Process (Human + Tool)

I started by brain storming my idea on a sticky note, planning what I wanted the questions to be and what I wanted the flow to follow. I made the idea of the end screen for each story option. With this I asked chat to create a code based off of the existing structure in separate files. GenAI had struggles with making the code flow from separate files so I had to separate the code and edit the index on my own. They style and colors I also went into the code to fix since there weren't many visual design aspects. I added a box to the back of the buttons to make them look like so. I asked chat to add a hover option and I changed all the colors of the hover and background screens.

Verification & Judgement

I made sure to playtest after every line that I changed to make sure that the code was running smoothly. Something that did not work was trying to understand how to get all the separate flies working together but I was able to fix that issue manually.

Appendix

Prompt:

I have to make an interactive story code. This is the basic code i have to start with:

You said:

```
// NOTE: Do NOT add setup() or draw() in this file
// setup() and draw() live in main.js
// This file only defines:
// 1) drawWin() → what the win screen looks like
// 2) input handlers → how the player returns to the start screen
//
// This file is intentionally very similar to lose.js.
// The goal is to show that win/lose screens are often
```

```

// simple "end states" with minimal logic.

// -----
// Main draw function for win screen
// -----
// drawWin() is called from main.js
// only when currentScreen === "win"
function drawWin() {
  // Green-tinted background to communicate success
  background(200, 255, 200);

  fill(0);
  textAlign(CENTER, CENTER);

  // Main success message
  textSize(40);
  text("You Win!", width / 2, 300);

  // Instruction text
  textSize(20);
  text("Click or press R to return to Start.", width / 2, 360);
}

// -----
// Mouse input for win screen
// -----
// Any mouse click returns the player to the start screen
function winMousePressed() {
  currentScreen = "start";
}

// -----
// Keyboard input for win screen
// -----
// R is commonly used for "restart" in games
function winKeyPressed() {
  if (key === "r" || key === "R") {
    currentScreen = "start";
  }
}

```

I was thinking that we could do a lunch time story where; the start screen displays a text that says "Lunch time!" and "click or press entre to start" thai would take you to a second screen where the question asks "you are hungry! What do you want to eat?"

and give the player 2 choices; hot dog and pasta once they choose no matter what it take them to a second screen where the question says “what do you want to drink?” and here give the player 2 options water and soda, if they pick soda make it so that the end screen says “you exploded” and “press r to restart” and if they choose water take them to a screen that says “you are full!” and “press r to restart” i need the game states to be on separate folders but are connected how would i approach this

Response

Folder structure

You can separate your files by **screens/game states**:

```
/project
|  main.js           ← setup(), draw(), global state management
|  index.html
|
|  └─ /screens
|     │  start.js      ← starting screen
|     │  food.js       ← “What do you want to eat?” screen
|     │  drink.js      ← “What do you want to drink?” screen
|     │  full.js       ← “You are full” ending
|     └─ exploded.js   ← “You exploded” ending
```

This keeps each screen modular and easy to expand.

2 main.js – global game controller

This file handles **which screen is currently active** and forwards input/draw calls to the right screen.

```
let currentScreen = "start";

// Preload or import screens
// For simplicity, assume all screen files are linked in
index.html
// and define their draw / input functions

function setup() {
```

```

    createCanvas(600, 400);
}

function draw() {
    background(220);

    // Call the draw function based on currentScreen
    if (currentScreen === "start") drawStart();
    else if (currentScreen === "food") drawFood();
    else if (currentScreen === "drink") drawDrink();
    else if (currentScreen === "full") drawFull();
    else if (currentScreen === "exploded") drawExploded();
}

function mousePressed() {
    if (currentScreen === "start") startMousePressed();
    else if (currentScreen === "food") foodMousePressed();
    else if (currentScreen === "drink") drinkMousePressed();
    else if (currentScreen === "full") fullMousePressed();
    else if (currentScreen === "exploded") explodedMousePressed();
}

function keyPressed() {
    if (currentScreen === "start") startKeyPressed();
    else if (currentScreen === "food") foodKeyPressed();
    else if (currentScreen === "drink") drinkKeyPressed();
    else if (currentScreen === "full") fullKeyPressed();
    else if (currentScreen === "exploded") explodedKeyPressed();
}

```

✓ This pattern lets you **add or change screens** easily without touching main.js much.

3 Example screen – start.js

```

function drawStart() {
    background(150, 200, 255);
    fill(0);
}

```

```

    textAlign(CENTER, CENTER);
    textSize(40);
    text("Lunch Time!", width / 2, 150);
    textSize(20);
    text("Click to start your lunch adventure", width / 2, 250);
}

function startMousePressed() {
    currentScreen = "food"; // Move to food choice screen
}

function startKeyPressed() {
    if (key === "Enter") currentScreen = "food";
}

```

4 Food choice screen – food.js

```

function drawFood() {
    background(255, 230, 200);
    textAlign(CENTER, CENTER);
    textSize(30);
    text("You're hungry! What do you want to eat?", width / 2,
100);

    textSize(24);
    text("1. Hotdog", width / 2, 200);
    text("2. Pasta", width / 2, 250);
}

function foodMousePressed() {
    // You can detect clicks roughly by Y position
    if (mouseY > 180 && mouseY < 220) currentScreen = "drink"; //
Hotdog
    else if (mouseY > 230 && mouseY < 270) currentScreen =
"drink"; // Pasta
}

```

```
function foodKeyPressed() {  
  if (key === '1') currentScreen = "drink";  
  else if (key === '2') currentScreen = "drink";  
}
```

5 Drink choice screen – drink.js

```
function drawDrink() {  
  background(200, 255, 200);  
  textAlign(CENTER, CENTER);  
  textSize(30);  
  text("What do you want to drink?", width / 2, 100);  
  
  textSize(24);  
  text("1. Water", width / 2, 200);  
  text("2. Soda", width / 2, 250);  
}  
  
function drinkMousePressed() {  
  if (mouseY > 180 && mouseY < 220) currentScreen = "full";  
  // Water  
  else if (mouseY > 230 && mouseY < 270) currentScreen =  
  "exploded"; // Soda  
}  
  
function drinkKeyPressed() {  
  if (key === '1') currentScreen = "full";  
  else if (key === '2') currentScreen = "exploded";  
}
```

6 Ending screens – full.js & exploded.js

full.js:

```
function drawFull() {  
  background(150, 255, 200);
```

```

    textAlign(CENTER, CENTER);
    textSize(40);
    text("You are full!", width / 2, 200);
    textSize(20);
    text("Click or press R to return to start.", width / 2, 300);
}

```

```

function fullMousePressed() { currentScreen = "start"; }
function fullKeyPressed() { if (key === "r" || key === "R")
currentScreen = "start"; }

```

exploded.js:

```

function drawExploded() {
    background(255, 100, 100);
    textAlign(CENTER, CENTER);
    textSize(40);
    text("You exploded!", width / 2, 200);
    textSize(20);
    text("Click or press R to return to start.", width / 2, 300);
}

```

```

function explodedMousePressed() { currentScreen = "start"; }
function explodedKeyPressed() { if (key === "r" || key === "R")
currentScreen = "start"; }

```

Prompt:

when i click start a grey screen appars and i cannot do anything else

Response:

Ah! That's a **classic problem** when setting up modular screens in p5.js. The gray screen usually happens because **the draw function is running, but your draw functions in the imported screen files aren't actually being called or aren't defined correctly**. Let's debug carefully.

Make sure all screen files are loaded

In your **index.html**, you must include all the JS files **after p5.js** and in the correct order (so that **main.js** runs last, or at least after the screen files):


```
<script
src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.6.0/p5.js"><
/script>

<!-- Screens -->
<script src="screens/start.js"></script>
<script src="screens/food.js"></script>
<script src="screens/drink.js"></script>
<script src="screens/full.js"></script>
<script src="screens/exploded.js"></script>

<!-- Main controller -->
<script src="main.js"></script>
```

If you don't include the screen files, `drawStart()` etc. are undefined → nothing happens → gray canvas.

2 Make sure your draw functions exist

Example from `start.js`:

```
function drawStart() {
  background(150, 200, 255); // Not gray
  fill(0);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("Lunch Time!", width / 2, 150);
  textSize(20);
  text("Click to start your lunch adventure", width / 2, 250);
}

function startMousePressed() {
  currentScreen = "food"; // Must match exactly the string in
  main.js
}

function startKeyPressed() {
```

```
    if (key === "Enter") currentScreen = "food";  
  }
```

Check these points:

- `drawStart()` is spelled correctly in **both start.js and main.js**.
- `currentScreen` strings match exactly: `"start"`, `"food"`, etc.

Any typo will break it silently and show just a blank/gray canvas.

3 Make sure `main.js` is forwarding input correctly

Your `mousePressed` in `main.js` must look like this:

```
function mousePressed() {  
  if (currentScreen === "start") startMousePressed();  
  else if (currentScreen === "food") foodMousePressed();  
  else if (currentScreen === "drink") drinkMousePressed();  
  else if (currentScreen === "full") fullMousePressed();  
  else if (currentScreen === "exploded") explodedMousePressed();  
}
```

- If `startMousePressed()` doesn't exist or `currentScreen` doesn't match `"start"`, nothing happens → gray screen.
-

4 Make sure your `draw()` calls the right function

In `main.js`:

```
function draw() {  
  if (currentScreen === "start") drawStart();  
  else if (currentScreen === "food") drawFood();  
  else if (currentScreen === "drink") drawDrink();  
}
```

```

else if (currentScreen === "full") drawFull();
else if (currentScreen === "exploded") drawExploded();
}

```

- Even one typo (`drawStart` vs `drawstart`) will fail.
- Make sure **all screen files are loaded before main.js**, otherwise these functions are undefined → gray screen.

5 Debugging tip

Add this **temporary debug line** in `draw()`:

```
console.log("Current screen:", currentScreen);
```

- When you click start, the console should show:
"Current screen: food"
- If it still says "start" after clicking, your `startMousePressed()` isn't firing → check mousePressed wiring.

Prompt:

could you give me all the code in one page

Response:

```

// ----- GLOBAL VARIABLES -----
let currentScreen = "start";

// ----- P5 SETUP -----
function setup() {
  createCanvas(600, 400);
}

// ----- P5 DRAW -----
function draw() {
  background(220);

```

```

// Call the correct screen
if (currentScreen === "start") drawStart();
else if (currentScreen === "food") drawFood();
else if (currentScreen === "drink") drawDrink();
else if (currentScreen === "full") drawFull();
else if (currentScreen === "exploded") drawExploded();
}

// ----- MOUSE INPUT -----
function mousePressed() {
  if (currentScreen === "start") startMousePressed();
  else if (currentScreen === "food") foodMousePressed();
  else if (currentScreen === "drink") drinkMousePressed();
  else if (currentScreen === "full") endMousePressed();
  else if (currentScreen === "exploded") endMousePressed();
}

// ----- KEYBOARD INPUT -----
function keyPressed() {
  if (currentScreen === "start") startKeyPressed();
  else if (currentScreen === "food") foodKeyPressed();
  else if (currentScreen === "drink") drinkKeyPressed();
  else if (currentScreen === "full") endKeyPressed();
  else if (currentScreen === "exploded") endKeyPressed();
}

// ----- START SCREEN -----
function drawStart() {
  background(150, 200, 255);
  fill(0);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("Lunch Time!", width / 2, 150);
  textSize(20);
  text("Click to start your lunch adventure", width / 2, 250);
}

function startMousePressed() {
  currentScreen = "food";
}

function startKeyPressed() {
  if (key === "Enter") currentScreen = "food";
}

```

```

}

// ----- FOOD CHOICE SCREEN -----
function drawFood() {
  background(255, 230, 200);
  textAlign(CENTER, CENTER);
  textSize(30);
  text("You're hungry! What do you want to eat?", width / 2, 100);

  textSize(24);
  text("Hotdog", width / 2, 200);
  text("Pasta", width / 2, 250);
}

function foodMousePressed() {
  if (mouseY > 180 && mouseY < 220) currentScreen = "drink"; // Hotdog
  else if (mouseY > 230 && mouseY < 270) currentScreen = "drink"; // Pasta
}

function foodKeyPressed() {
  if (key === '1') currentScreen = "drink";
  else if (key === '2') currentScreen = "drink";
}

// ----- DRINK CHOICE SCREEN -----
function drawDrink() {
  background(200, 255, 200);
  textAlign(CENTER, CENTER);
  textSize(30);
  text("What do you want to drink?", width / 2, 100);

  textSize(24);
  text("Water", width / 2, 200);
  text("Soda", width / 2, 250);
}

function drinkMousePressed() {
  if (mouseY > 180 && mouseY < 220) currentScreen = "full"; // Water
  else if (mouseY > 230 && mouseY < 270) currentScreen = "exploded"; // Soda
}

function drinkKeyPressed() {
  if (key === '1') currentScreen = "full";

```

```

    else if (key === '2') currentScreen = "exploded";
}

// ----- FULL SCREEN -----
function drawFull() {
  background(150, 255, 200);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("You are full!", width / 2, 200);
  textSize(20);
  text("Click or press R to return to start.", width / 2, 300);
}

// ----- EXPLODED SCREEN -----
function drawExploded() {
  background(255, 100, 100);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("You exploded!", width / 2, 200);
  textSize(20);
  text("Click or press R to return to start.", width / 2, 300);
}

// ----- END SCREENS INPUT -----
function endMousePressed() {
  currentScreen = "start";
}

function endKeyPressed() {
  if (key === "r" || key === "R") currentScreen = "start";
}

```

Prompt:

i am using visuall stuido code how do i make all those seprate codes connect so that the prgram works

Response:

Folder Structure in VS Code

Let's organize your project like this:

`lunchtime-game/`

```
| index.html
| main.js
|
└─ screens/
    start.js
    food.js
    drink.js
    full.js
    exploded.js
```

Make sure the folder names match exactly (**screens** lowercase).

2 index.html – link all JS files

In **VS Code**, open `index.html` and include the scripts **in this order**:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Lunchtime Adventure</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.6.0/p5.js"><
/script>

  <!-- Load all screens first -->
  <script src="screens/start.js"></script>
  <script src="screens/food.js"></script>
  <script src="screens/drink.js"></script>
  <script src="screens/full.js"></script>
  <script src="screens/exploded.js"></script>

  <!-- Then load main.js last -->
  <script src="main.js"></script>
</head>
<body>
</body>
```

</html>

Why order matters: `main.js` calls functions like `drawStart()`. If `start.js` hasn't loaded yet, the function doesn't exist → gray screen.

3 main.js – global controller

This file does **not need to change**. It simply routes to the right screen based on `currentScreen`:

```
let currentScreen = "start";

function setup() {
  createCanvas(600, 400);
}

function draw() {
  background(220);

  if (currentScreen === "start") drawStart();
  else if (currentScreen === "food") drawFood();
  else if (currentScreen === "drink") drawDrink();
  else if (currentScreen === "full") drawFull();
  else if (currentScreen === "exploded") drawExploded();
}

function mousePressed() {
  if (currentScreen === "start") startMousePressed();
  else if (currentScreen === "food") foodMousePressed();
  else if (currentScreen === "drink") drinkMousePressed();
  else if (currentScreen === "full") fullMousePressed();
  else if (currentScreen === "exploded") explodedMousePressed();
}

function keyPressed() {
  if (currentScreen === "start") startKeyPressed();
  else if (currentScreen === "food") foodKeyPressed();
}
```



```
else if (currentScreen === "drink") drinkKeyPressed();
else if (currentScreen === "full") fullKeyPressed();
else if (currentScreen === "exploded") explodedKeyPressed();
}
```

4 screens – unique function names

Each screen file must **define its own draw and input functions**, like this:

screens/start.js

```
function drawStart() {
  background(150, 200, 255);
  fill(0);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("Lunch Time!", width / 2, 150);
  textSize(20);
  text("Click to start your lunch adventure", width / 2, 250);
}
```

```
function startMousePressed() { currentScreen = "food"; }
function startKeyPressed() { if(key==="Enter")
currentScreen="food"; }
```

screens/food.js

```
function drawFood() {
  background(255, 230, 200);
  textAlign(CENTER, CENTER);
  textSize(30);
  text("You're hungry! What do you want to eat?", width/2, 100);
  textSize(24);
  text("Hotdog", width/2, 200);
  text("Pasta", width/2, 250);
}
```

```
function foodMousePressed() { currentScreen="drink"; }  
function foodKeyPressed() { if(key==='1' || key==='2')  
currentScreen="drink"; }
```

screens/drink.js

```
function drawDrink() {  
    background(200, 255, 200);  
    textAlign(CENTER, CENTER);  
    textSize(30);  
    text("What do you want to drink?", width/2, 100);  
    textSize(24);  
    text("Water", width/2, 200);  
    text("Soda", width/2, 250);  
}  
  
function drinkMousePressed() {  
    if(mouseY>180 && mouseY<220) currentScreen="full";  
    else if(mouseY>230 && mouseY<270) currentScreen="exploded";  
}  
function drinkKeyPressed() {  
    if(key==='1') currentScreen="full";  
    else if(key==='2') currentScreen="exploded";  
}
```

screens/full.js

```
function drawFull() {  
    background(150, 255, 200);  
    textAlign(CENTER, CENTER);  
    textSize(40);  
    text("You are full!", width/2, 200);  
    textSize(20);  
    text("Click or press R to return to start.", width/2, 300);  
}  
  
function fullMousePressed() { currentScreen="start"; }
```

```
function fullKeyPressed() { if(key==="r"||key==="R")
currentScreen="start"; }
```

screens/explored.js

```
function drawExplored() {
  background(255, 100, 100);
  textAlign(CENTER, CENTER);
  textSize(40);
  text("You explored!", width/2, 200);
  textSize(20);
  text("Click or press R to return to start.", width/2, 300);
}
```

```
function exploredMousePressed() { currentScreen="start"; }
function exploredKeyPressed() { if(key==="r"||key==="R")
currentScreen="start"; }
```

5 How to run in VS Code

1. Open the folder in VS Code.
2. Install the **Live Server extension** (if you don't have it).
3. Right-click `index.html` → **Open with Live Server**.
4. The game should load in the browser.
5. Click / use keys to test transitions between screens.

⚠ If you just open the HTML file directly without a server, sometimes scripts don't load properly in Chrome due to **CORS/security restrictions**. Live Server fixes this.

Prompt:

can you add a hover effect over the options hotdog and pasta and water and soda

Response:

Update `food.js` (hotdog/pasta)

```

function drawFood() {
  background(255, 230, 200);
  textAlign(CENTER, CENTER);
  textSize(30);
  fill(0);
  text("You're hungry! What do you want to eat?", width/2, 100);

  textSize(24);

  // Hotdog hover
  if(mouseX > width/2 - 50 && mouseX < width/2 + 50 && mouseY > 185 && mouseY <
215) {
    fill(255, 0, 0); // red on hover
  } else {
    fill(0);
  }
  text("Hotdog (1)", width/2, 200);

  // Pasta hover
  if(mouseX > width/2 - 50 && mouseX < width/2 + 50 && mouseY > 235 && mouseY <
265) {
    fill(255, 0, 0); // red on hover
  } else {
    fill(0);
  }
  text("Pasta (2)", width/2, 250);
}

function foodMousePressed() {
  if(mouseY > 185 && mouseY < 215) currentScreen = "drink"; // Hotdog
  else if(mouseY > 235 && mouseY < 265) currentScreen = "drink"; // Pasta
}

function foodKeyPressed() {
  if(key==='1' || key==='2') currentScreen = "drink";
}

```

✓ What changed:

- We check `mouseX` and `mouseY` to see if the mouse is over the text.
- If yes, we set `fill(255,0,0)` (red) to highlight it.

- Otherwise, it stays black.

2 Update drink.js (water/soda)

```
function drawDrink() {
  background(200, 255, 200);
  textAlign(CENTER, CENTER);
  textSize(30);
  fill(0);
  text("What do you want to drink?", width/2, 100);
  textSize(24);

  // Water hover
  if(mouseX > width/2 - 50 && mouseX < width/2 + 50 && mouseY > 185 && mouseY <
215) {
    fill(255, 0, 0);
  } else {
    fill(0);
  }
  text("Water (1)", width/2, 200);

  // Soda hover
  if(mouseX > width/2 - 50 && mouseX < width/2 + 50 && mouseY > 235 && mouseY <
265) {
    fill(255, 0, 0);
  } else {
    fill(0);
  }
  text("Soda (2)", width/2, 250);
}

function drinkMousePressed() {
  if(mouseY > 185 && mouseY < 215) currentScreen = "full"; // Water
  else if(mouseY > 235 && mouseY < 265) currentScreen = "exploded"; // Soda
}

function drinkKeyPressed() {
  if(key==='1') currentScreen="full";
  else if(key==='2') currentScreen="exploded";
}
```

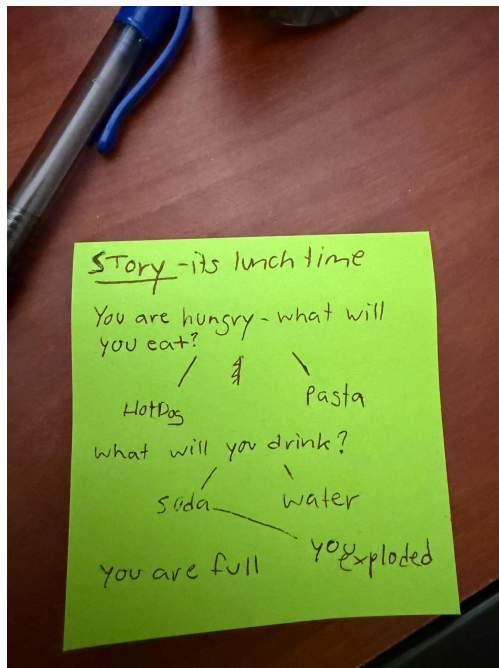
✓ Notes:

1. The numbers 185–215 and 235–265 are vertical ranges around your text Y positions (200 and 250). You can adjust if you change font size.
2. The horizontal range $\text{width}/2 - 50 \rightarrow \text{width}/2 + 50$ is a simple guess for text width. For better precision, you can use `textWidth()`:

```
let txt = "Hotdog (1)";  
let w = textWidth(txt);  
if(mouseX > width/2 - w/2 && mouseX < width/2 + w/2) { ... }
```

Role-Based Process Evidence

Name: Mia Krzyształowicz



can you add a hover effect over the options hotdog and pasta and water and soda

Absolutely! Since your game currently draws the options on the `p5 canvas` using `text()`, we can make a **hover effect** by checking if the mouse is over the text and changing its color or style.

I'll show you how to do it for **Hotdog/Pasta** and **Water/Soda**. This will work in your modular setup.

Update `food.js` (hotdog/pasta)

```
javascript Copy code  
  
function drawFood() {  
  background(255, 230, 200);  
  textAlign(CENTER, CENTER);  
  textSize(30);  
  fill(0);  
  text("You're hungry! What do you want to eat?", width/2, 100);
```